Saronic Interview Project

Saronic

Contents

Saronic Interview Project **2**

1 Project 1 — Distributed Key-Value Store **3**

2 Project 2 — Fleet Simulation Engine **4**

Saronic Interview Project

Welcome, and thanks for taking the time to work through this. You may choose **either one** of the projects described below. Pick whichever plays best to your strengths and interests:

- **Project 1 — Distributed Key-Value Store (Rust):** Build an etcd-like key-value store in Rust with a small HTTP client API, aiming for consistency and partition tolerance.
- **Project 2 — Fleet Simulation Engine:** Build a discrete-time simulation engine for a fleet of vessels driven by external commands over a simple stdin/stdout JSON protocol.

What to submit

Whichever you choose, please include:

- All of your code.
- A way to run and test it.
- A design doc.

AI tooling may be used throughout **except** for the writing of the design doc itself.

Chapter 1

Project 1 — Distributed Key-Value Store

Build a distributed key value store similar to etcd in Rust.

You are only allowed to use the following dependencies:

- `tokio`
- `axum`
- `serde` / `serde_json`
- `tracing` / `tracing_subscriber`

The server should support the following client API:

- **`GET /{key}`** — `200` response with the JSON object stored; if not found return a `404`.
- **`PUT /{key}`** — Accepts a JSON object and returns `201` if successfully written.
- **`DELETE /{key}`** — Delete a key and return `204` if successful.

etcd provides a consistent and partition tolerant system (CAP Theorem) so you should aim for the same goals.

The final submission should have all of the code for the project including a way to run/test the server. There should also be a design doc. AI can be used for every part of this project except for the writing of the design doc.

Chapter 2

Project 2 — Fleet Simulation Engine

Base Problem: Vessel Simulation Engine

Background

Discrete-time simulation engine for a fleet of vessels. It is driven entirely by external commands. Its purpose is to track and report vessel state at each point in time.

I/O Model

Reads newline-delimited JSON from **stdin**, writes to **stdout**. Each command produces exactly one line of output, processed in order.

Every response is either `{"status": "ok", "payload": <WorldState>}` or `{"status": "error", "payload": {"message": "<reason>"}}`.

Behavioral Rules

Execution Order

Commands take effect in the order received. **Planned State** is updated before the response is written.

Committed State vs. Planned State

- **Committed State**: The state of the world at a tick. It is a snapshot of the **planned state** when `tick` is called.
- **Planned State**: The expected state of the world given the commands since the last `tick` applied to the **committed state**. This updates immediately with each command and becomes the new **committed state** when `tick` is called.

For example, if a vessel is `idle` at the last **committed state**, the `set_destination` can be called on that vessel as many times as the user wants. Each `set_destination` command for the vessel overrides the last `set_destination` command and updates the **planned state**. Only once `tick` is called, does the **planned state** become the new **committed state**, and further `set_destination` will fail since the vessel is in `transit`.

Tick semantics

- Each `tick` command advances the simulation by exactly one time step.
- A `set_destination` issued at tick `T` sets `arrives_at_tick = T + travel_times[current_location][destination]`.
- **A vessel that arrives at tick `T` reports `idle` at tick `T`.** It does not report `transit` on the tick it arrives.

- A vessel reporting `idle` may be commanded in the same tick it arrives.

Commands

`init` — Must be the first command. Returns world state at tick 0. Any other command before `init`, or a second `init`, returns an error.

```
{"command": "init", "parameters": {"travel_times": [[0,4,7],[4,0,3],[7,3,0]], "vessels": [{"id": "v1", "location":
```

```
{"status": "ok", "payload": {"tick": 0, "vessels": {"v1": {"state": "idle", "location": 0}, "v2": {"state": "idle"
```

- `travel_times`: N×N integer matrix. `travel_times[i][j]` is ticks to travel from i to j. Diagonal is 0; all off-diagonal values are positive integers.
- `vessels`: list of `{id, location}`. Each vessel starts idle at the given location index.

`tick` — Advances the simulation by one time step.

```
{"command": "tick", "parameters": {}}
```

```
{"status": "ok", "payload": {"tick": 1, "vessels": {"v1": {"state": "transit", "from": 0, "to": 1, "arrives_at_tic
```

`set_destination` — Commands a vessel to travel to a location. Valid only if the **committed state** is `idle`. Rejected if vessel is in transit, `vessel_id` is unknown, or destination is out of range.

Left to the implementer: What happens if a user commands `set_destination` for the vessel's current location?

```
{"command": "set_destination", "parameters": {"vessel_id": "v1", "destination": 1}}
```

```
{"status": "ok", "payload": {"tick": 0, "vessels": {"v1": {"state": "transit", "from": 0, "to": 1, "arrives_at_tic
```

On failure:

```
{"status": "error", "payload": {"message": "vessel v1 is not idle"}}
```

- `destination`: valid location index (0 to N-1)

`get_state` — Returns **planned state**.

```
{"command": "get_state", "parameters": {}}
```

```
{"status": "ok", "payload": {"tick": 0, "vessels": {"v1": {"state": "transit", "from": 0, "to": 1, "arrives_at_tic
```

Example

Setup: `travel_times = [[0,4,7],[4,0,3],[7,3,0]]`. v1 starts at location 0, v2 starts at location 2.

`stdin` `stdout`

```
{"command":"init","parameters":{"travel_times":[[0,4,7],[4,0,3],[7,3,0]],"vessels":[{"id":"v1","location":0},{"id"
  → {"status":"ok","payload":{"tick":0,"vessels":{"v1":{"state":"idle","location":0},"v2":{"state":"idle","locatio

# send v1 to location 2, then override to location 1 — both valid before tick
{"command":"set_destination","parameters":{"vessel_id":"v1","destination":2}}
  → {"status":"ok","payload":{"tick":0,"vessels":{"v1":{"state":"transit","from":0,"to":2,"arrives_at_tick":7},"v2
{"command":"set_destination","parameters":{"vessel_id":"v1","destination":1}}
```

```
  → {"status":"ok","payload":{"tick":0,"vessels":{"v1":{"state":"transit","from":0,"to":1,"arrives_at_tick":4},"v2

# tick commits destination 1
{"command":"tick","parameters":{}}
  → {"status":"ok","payload":{"tick":1,"vessels":{"v1":{"state":"transit","from":0,"to":1,"arrives_at_tick":4},"v2

# rejected — v1 is committed to transit since last tick
{"command":"set_destination","parameters":{"vessel_id":"v1","destination":2}}
  → {"status":"error","payload":{"message":"vessel v1 is not idle"}}

# [ticks 2–3 omitted — v1 still in transit]
# tick 4: v1 arrives at location 1
{"command":"tick","parameters":{}}
  → {"status":"ok","payload":{"tick":4,"vessels":{"v1":{"state":"idle","location":1},"v2":{"state":"idle","locatio

# v1 arrived — can be commanded immediately
{"command":"set_destination","parameters":{"vessel_id":"v1","destination":2}}
  → {"status":"ok","payload":{"tick":4,"vessels":{"v1":{"state":"transit","from":1,"to":2,"arrives_at_tick":7},"v2
```

Follow-on 1: Cargo and Dock Operations

Change from base

We add a new concept of cargo and docks to the sim. Each location has a dock which maintains an inventory of cargo and has `dock_throughput`, which is the maximum number of cargo operations that a dock can process in a single tick. Cargo is simply represented by its destination, there is no distinction between two cargo with the same destination.

Vessels load from dock inventory at their current location and can unload at any dock. All world state responses should now also report dock `cargo` inventories and vessel `cargo` inventories.

Cargo enters via a new `cargo_manifest` field on `init` and on `tick`, adding units to dock inventory.

Five new commands manage dock operations:

- **`dock`** — Parks the vessel at its current location and sets **planned state** to `docked`. Valid if the committed state is `idle`. Overwritable by `set_destination` before the next `tick`. Similarly, a pending `set_destination` can be overwritten by `dock` before the next `tick`.
  - Left to the implementer: What happens if a user commands `dock` after `dock`?
  - Left to the implementer: What happens if a user commands `undock` after `dock`?
- **`undock`** — Returns vessel to `idle` at current location. Valid if the committed state is `docked`.
  - Left to the implementer: What happens if a user commands `undock` after `undock`?
  - Left to the implementer: What happens if a user commands `dock` after `undock`?
- **`load`** — Moves one cargo from dock → vessel and updates **planned state** inventory. Valid when **committed state** is `docked` and inventory exists.
- **`unload`** — Moves one cargo from vessel → dock and updates **planned state** inventory. Valid when **committed state** is `docked` and inventory exists.
- **`swap`** — Simultaneous load + unload. Valid when **committed state** is `docked` and inventory exists.

Left to the implementer: What happens if a user commands `undock` after a `load`/`unload`/`swap` command? What happens if a user commands `load`/`unload`/`swap` after an `undock` command?

Dock Operation Rules

Dock Throughput

`dock_throughput[location]` limits the total number of cargo operations (`load`, `unload`, `swap` each count as one) that can be run across all vessels at that dock between ticks. Each `load`/`unload`/`swap` command is checked against this limit when issued; the command is rejected if the new pending state cannot be achieved within the `dock_throughput` limit.

Left to the implementer: What happens if a user calls `load` and `unload` on the same cargo? How many operations does this count as?

Commands

`init` — Now requires `dock_throughput`. `cargo_manifest` is optional and initializes docks with cargo inventory.

```
{"command": "init", "parameters": {"travel_times": [[0,4,7],[4,0,3],[7,3,0]], "dock_throughput": [2,4,1], "vessels
```

```
{"status": "ok", "payload": {"tick": 0, "vessels": {"v1": {"state": "idle", "location": 0, "cargo": []}, "v2": {"s
```

- `dock_throughput`: N-length integer array. `dock_throughput[i]` is the max operations across all vessels at dock i per tick. All values positive.
- `cargo_manifest`: optional. Each entry is `{location, destination, quantity}` — `location` and `destination` both must be valid location indices. `quantity` must be a positive integer. If any entry is out of range, the command is rejected.

`tick` — `cargo_manifest` is optional and adds cargo inventory to the newly **committed state**.

```
{"command": "tick", "parameters": {"cargo_manifest": [{"location": 0, "destination": 2, "quantity": 1}, {"location
```

```
{"status": "ok", "payload": {"tick": 1, "vessels": {"v1": {"state": "idle", "location": 0, "cargo": []}, "v2": {"s
```

- `cargo_manifest`: optional. Each entry is `{location, destination, quantity}` — `location` and `destination` both must be valid location indices. `quantity` must be a positive integer. If any entry is out of range, the command is rejected.

`dock` — Parks the vessel at its current location and sets **planned state** to `docked`. Valid if the committed state is `idle`. Overwritable by `set_destination` before the next `tick`.

tick = 1, v1 **committed state** is `idle` at location `0`:

```
{"command": "dock", "parameters": {"vessel_id": "v1"}}
```

```
{"status": "ok", "payload": {"tick": 1, "vessels": {"v1": {"state": "docked", "location": 0, "cargo": []}, "v2": {
```

On failure:

```
{"status": "error", "payload": {"message": "vessel v1 is not idle"}}
```

`undock` — Returns vessel to `idle` at current location. Valid if the committed state is `docked`.

tick = 3, v1 **committed state** is `docked` at location `0`:

```
{"command": "undock", "parameters": {"vessel_id": "v1"}}
```

```
{"status": "ok", "payload": {"tick": 3, "vessels": {"v1": {"state": "idle", "location": 0, "cargo": [{"destination
```

On failure:

```
{"status": "error", "payload": {"message": "vessel v1 is not docked"}}
```

`load` — Moves one cargo from dock → vessel and updates **planned state** inventory. Valid when **committed state** is `docked` and inventory exists.

```
{"command": "load", "parameters": {"vessel_id": "v1", "destination": 2}}
```

```
{"status": "ok", "payload": {"tick": 2, "vessels": {"v1": {"state": "docked", "location": 0, "cargo": [{"destinati
```

On failure:

```
{"status": "error", "payload": {"message": "vessel v1 is not docked"}}
```

```
{"status": "error", "payload": {"message": "transfer of 3 operations exceeds dock throughput of 2 at location 0"}}
```

- `destination`: destination of the cargo unit to load from dock inventory

---

`unload` — Moves one cargo from vessel → dock and updates **planned state** inventory. Valid when **committed state** is `docked` and inventory exists. If the cargo's destination matches the vessel's current location, the cargo is **delivered** and removed from the simulation rather than added to the dock inventory.

```
{"command": "unload", "parameters": {"vessel_id": "v1", "destination": 2}}
```

```
{"status": "ok", "payload": {"tick": 12, "vessels": {"v1": {"state": "docked", "location": 2, "cargo": [{"destinat
```

On failure:

```
{"status": "error", "payload": {"message": "vessel v1 is not docked"}}
```

- `destination`: destination of the cargo unit to remove from the vessel

---

`swap` — Simultaneous load + unload. Valid when **committed state** is `docked` and inventory exists.

tick = 2, v1 **committed state** is `docked` at location `0` with cargo `[{"destination": 2, "quantity": 1}]`; dock 0 has `[{"destination": 1, "quantity": 1}]`:

```
{"command": "swap", "parameters": {"vessel_id": "v1", "load_destination": 1, "unload_destination": 2}}
```

```
{"status": "ok", "payload": {"tick": 2, "vessels": {"v1": {"state": "docked", "location": 0, "cargo": [{"destinati
```

- `load_destination`: destination of the cargo unit to load from dock inventory
- `unload_destination`: destination of the cargo unit to remove from the vessel

---

`get_state` — Response now includes dock `cargo` in world state and each vessel includes `cargo`.

```
{"command": "get_state", "parameters": {}}
```

```
{"status": "ok", "payload": {"tick": 3, "vessels": {"v1": {"state": "idle", "location": 0, "cargo": [{"destination
```

Example

Setup: `travel_times = [[0,4,7],[4,0,3],[7,3,0]]`, `dock_throughput = [2,4,1]`. v1 at location 0, v2 at location 2. Tick 1 delivers 1 unit to destination 2 and 1 unit to destination 1 at location 0.

```
stdin                                                                    stdout
-------------------------------------------------------------------------------
{"command":"init","parameters":{"travel_times":[[0,4,7],[4,0,3],[7,3,0]],"dock_throughput":[2,4,1],"vessels":[{"id
  → {"status":"ok","payload":{"tick":0,"vessels":{"v1":{"state":"idle","location":0,"cargo":[]},"v2":{"state":"idl

# tick 1: 1 unit to dest 2 and 1 unit to dest 1 arrive at location 0
{"command":"tick","parameters":{"cargo_manifest":[{"location":0,"destination":2,"quantity":1},{"location":0,"desti
  → {"status":"ok","payload":{"tick":1,"vessels":{"v1":{"state":"idle","location":0,"cargo":[]},"v2":{"state":"idl

# dock v1 at location 0 — planned state docked, committed still idle
{"command":"dock","parameters":{"vessel_id":"v1"}}
  → {"status":"ok","payload":{"tick":1,"vessels":{"v1":{"state":"docked","location":0,"cargo":[]},"v2":{"state":"i

# tick commits the dock
{"command":"tick","parameters":{}}
  → {"status":"ok","payload":{"tick":2,"vessels":{"v1":{"state":"docked","location":0,"cargo":[]},"v2":{"state":"i

# load both units (2 ops, throughput=2) — planned state updates immediately
{"command":"load","parameters":{"vessel_id":"v1","destination":2}}
  → {"status":"ok","payload":{"tick":2,"vessels":{"v1":{"state":"docked","location":0,"cargo":[{"destination":2,"q
{"command":"load","parameters":{"vessel_id":"v1","destination":1}}
  → {"status":"ok","payload":{"tick":2,"vessels":{"v1":{"state":"docked","location":0,"cargo":[{"destination":2,"q

# tick 3: commits the loads
{"command":"tick","parameters":{}}
  → {"status":"ok","payload":{"tick":3,"vessels":{"v1":{"state":"docked","location":0,"cargo":[{"destination":2,"q

# undock v1 — planned state idle, committed still docked
{"command":"undock","parameters":{"vessel_id":"v1"}}
  → {"status":"ok","payload":{"tick":3,"vessels":{"v1":{"state":"idle","location":0,"cargo":[{"destination":2,"quan

# tick commits the undock
{"command":"tick","parameters":{}}
  → {"status":"ok","payload":{"tick":4,"vessels":{"v1":{"state":"idle","location":0,"cargo":[{"destination":2,"quan

# send v1 to location 2 — arrives at tick 4 + 7 = 11
{"command":"set_destination","parameters":{"vessel_id":"v1","destination":2}}
  → {"status":"ok","payload":{"tick":4,"vessels":{"v1":{"state":"transit","from":0,"to":2,"arrives_at_tick":11,"ca

# [ticks 5–10 omitted — v1 in transit]
# tick 11: v1 arrives at location 2
{"command":"tick","parameters":{}}
  → {"status":"ok","payload":{"tick":11,"vessels":{"v1":{"state":"idle","location":2,"cargo":[{"destination":2,"qua

# park v1 at location 2
{"command":"dock","parameters":{"vessel_id":"v1"}}
  → {"status":"ok","payload":{"tick":11,"vessels":{"v1":{"state":"docked","location":2,"cargo":[{"destination":2,"

# tick commits the dock
{"command":"tick","parameters":{}}
  → {"status":"ok","payload":{"tick":12,"vessels":{"v1":{"state":"docked","location":2,"cargo":[{"destination":2,"

# unload dest 2 at location 2 (delivery, 1 op, throughput=1) — planned state updates immediately
{"command":"unload","parameters":{"vessel_id":"v1","destination":2}}
```

```
  → {"status":"ok","payload":{"tick":12,"vessels":{"v1":{"state":"docked","location":2,"cargo":[{"destination":1,"

# tick 13: commits the unload
{"command":"tick","parameters":{}}
  → {"status":"ok","payload":{"tick":13,"vessels":{"v1":{"state":"docked","location":2,"cargo":[{"destination":1,"
```

Follow-on 2: Historical State Queries

Change from base

Adds querying of past tick **committed states**. All existing behavior is unchanged.

`init` records **committed state** at tick 0; each `tick` records after all transitions. `get_state_at(T)` returns that **committed state**. Intra-tick states are not queryable.

Commands

`get_state_at` — Returns the **committed state** at a past tick. Errors if `tick` > current tick or `tick` < 0.

```
{"command": "get_state_at", "parameters": {"tick": 5}}
```

```
{"status": "ok", "payload": {"tick": 5, "vessels": {"v1": {"state": "transit", "from": 0, "to": 2, "arrives_at_tic
```

On failure:

```
{"status": "error", "payload": {"message": "tick 99 is in the future (current tick: 36)"}}
```

- `tick`: must be <= current tick and >= 0

Example

Continuing from Follow-on 1 after tick 13. Current tick is 13.

```
stdin                                                                    stdout
-------------------------------------------------------------------------------
# query state at a past tick
{"command":"get_state_at","parameters":{"tick":1}}
  → {"status":"ok","payload":{"tick":1,"vessels":{"v1":{"state":"idle","location":0,"cargo":[]},"v2":{"state":"idl

# query current tick — identical to get_state at tick boundary
{"command":"get_state_at","parameters":{"tick":13}}
  → {"status":"ok","payload":{"tick":13,"vessels":{"v1":{"state":"docked","location":2,"cargo":[{"destination":1,"

# querying the future is an error
{"command":"get_state_at","parameters":{"tick":99}}
  → {"status":"error","payload":{"message":"tick 99 is in the future (current tick: 13)"}}

# simulation state is unchanged after historical queries
{"command":"get_state","parameters":{}}
  → {"status":"ok","payload":{"tick":13,"vessels":{"v1":{"state":"docked","location":2,"cargo":[{"destination":1,"
```